

OPSASSIST

AI-Powered Ticketing System

Project Documentation

TEAM MEMBERS:

Fanelesibonge Hlebani

Mufhumudzi Moshapo

Asisipho Mbobo

Lukholo Mabuto

Minenhle Sibiya

Project Overview	2
System Description	3
The Problem It Solves	3
Project Objectives.....	3
System Features.....	3
Core Ticketing	3
AI-Generated Communication.....	4
Notifications.....	4
Reporting & Analytics	4
Governance & Compliance	4
Workflow Automation	4
User Management	5
System Architecture	5
Architecture Diagram.....	6
Component Overview	6
How the Components Communicate	6
Browser (React SPA)	6
Cloudflare Worker (TanStack Start).....	6
Lovable AI Gateway	6
Resend	7
pg_cron scheduler	7
Communication Summary	7
Technologies Used	7
Database Design	8
Entity-Relationship Diagram (ERD) Explanation	9
System Workflow.....	10
AI Functionality	11
AI Ticket Classification & Prioritization.....	12
AI Response Generation	12
Predictive Insights.....	12
AI Governance & Risk Detection.....	12
Value to the IT Support Process	12
Challenges Encountered	12
Inefficient AI Prompting.....	13

Classification Accuracy & Keyword Overlap	13
Tone & Response Consistency	13
Dashboard Information Overload.....	13
Team Coordination	13
Lessons Learned.....	13
Future Improvements	14
Installation & Running the Project.....	15
References	15
Screenshots of the system.....	15

Project Overview

System Description

OpsAssist is an AI-powered business operations platform built by Team ByteBuilders over an eight-week AI Bootcamp programme. It allows employees to submit internal support requests through a simple web form, which the system automatically classifies into the correct department (HR, IT, Finance, or Operations) and assigns a priority level (High, Medium, or Low) using AI. From this foundation, the platform grew sprint by sprint into a full business operations assistant covering AI-generated responses, analytics and executive reporting, predictive forecasting, AI governance and compliance monitoring, and a configurable multi-stage approval workflow engine.

The Problem It Solves

Organizations typically receive a high volume of internal support tickets covering IT issues, HR queries, financial requests, and operational concerns. Traditionally, these are manually reviewed and routed by staff, which is slow, inconsistent, and prone to error — and provides management with little visibility into request volumes, response performance, or emerging risk. OpsAssist addresses this by automating classification and first-response drafting, surfacing operational trends through dashboards and reports, forecasting future workload, and enforcing accountable, auditable approval processes for requests that require sign-off before work can begin.

Project Objectives

- Automatically classify internal support requests into the correct department using AI
- Generate AI-assisted, tone-appropriate first responses to reduce manual communication effort
- Provide administrators with real-time analytics on ticket volume, categories, and response times
- Automate the conversion of operational data into executive-level, department-specific reports
- Forecast future ticket volume and workload surges to support proactive resource planning
- Introduce AI governance features — activity logging, transparency, risk detection, and compliance reporting
- Automate multi-stage approval workflows for requests requiring departmental sign-off
- Deliver a single, deployable, portfolio-ready platform demonstrating applied AI and full-stack development skills

System Features

The following features were implemented across the eight sprints of the project:

Core Ticketing

- Ticket logging — users submit a title and description through a web-based submission form
- AI-powered ticket categorization — tickets are automatically routed to HR, IT, Finance, or Operations

- AI-powered priority assignment — each ticket is automatically assigned a High, Medium, or Low priority based on urgency indicators in the request text
- Keyword-based fallback classification — if the AI service is unavailable, a heuristic keyword matcher classifies the ticket, so the system degrades gracefully rather than failing
- Ticket status tracking — tickets are tracked from submission through to resolution
- Admin dashboard — a password-protected view for administrators to view, filter, and search all submitted tickets

AI-Generated Communication

- AI-generated responses — automatically drafted, professional replies based on ticket category and content
- Tone control — responses can be generated in Formal, Friendly, or Urgent tones
- Category-based response templates — department-specific language and structure

Notifications

- Email notifications — transactional email (OTP account activation, password reset, and ticket lifecycle notifications) delivered via the Resend API

Reporting & Analytics

- Analytics dashboard — visualizes ticket volume, category distribution, and response time trends using charts and summary cards
- Automated executive reporting — dashboard insights are converted into downloadable, department-specific business reports (PDF export)
- Predictive insights — a forecasting engine estimates future ticket volume, identifies trend direction, detects day-of-week seasonality, and flags potential workload surges

Governance & Compliance

- AI activity logging — every AI-assisted action (classification, response generation, prediction, escalation) is logged with a timestamp, confidence score, and ticket reference
- AI transparency panel — displays AI involvement, generated category, suggested solution, and confidence score for each ticket
- Risk detection — automatically flags low-confidence classifications, conflicting predictions, and cases requiring human review, each assigned a Low/Medium/High risk level
- Compliance reporting — generates reports on AI activity statistics, classification accuracy, and human review counts

Workflow Automation

- Conditional approval workflows — configurable, multi-stage approval chains (e.g. Finance approval, then Manager approval) for requests that require sign-off before operational work begins

- Approval dashboard — approvers see only requests assigned to them and can Approve, Reject, or Request Information
- Workflow history — every workflow action is logged for a complete audit trail

User Management

- Role-based authentication — admin and user accounts, with OTP-based account activation and password reset
- Row Level Security — database access is scoped by user role via Supabase Row Level Security policies

System Architecture

Architecture Diagram

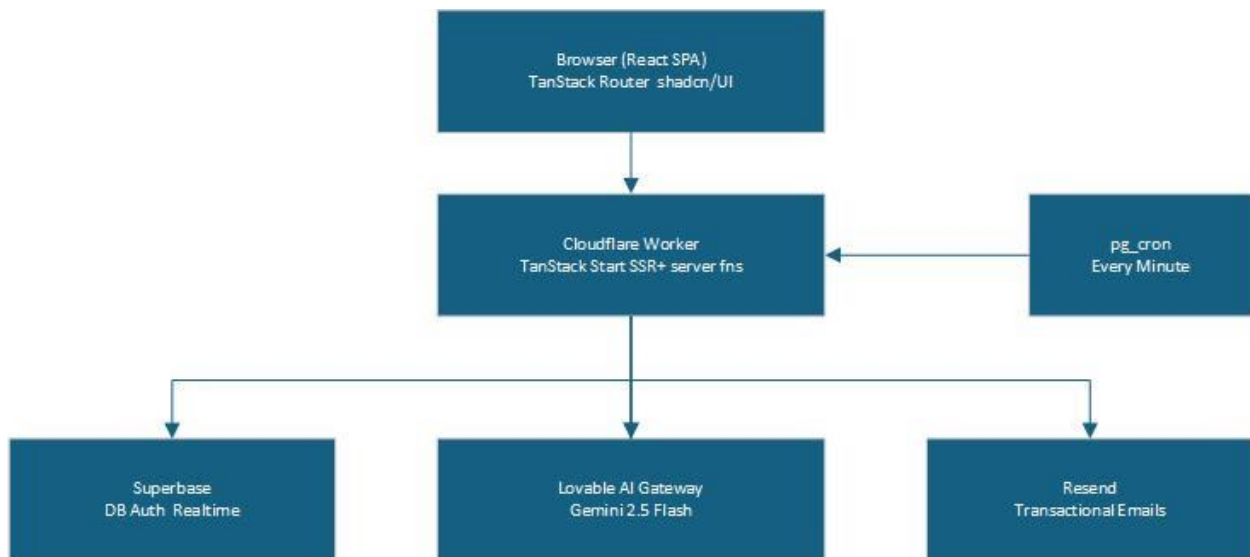


Figure 1: System workflow

Component Overview

OpsAssist follows a server-function-per-feature architecture. The frontend is a React single-page application that communicates directly with backend server functions — there is no separate REST or GraphQL API layer. Business logic is grouped into five modules: ticket submission and classification, user account management, workflow approvals, analytics and predictions, and compliance reporting.

How the Components Communicate

Browser (React SPA)

Built with React, TanStack Router, and shadcn/ui, client-rendered after server-side rendering. Two things happen directly from the browser, without going through the Worker:

- **Authentication** — the Supabase client SDK handles sign-in and session storage.
- **Realtime updates** — the browser subscribes to Supabase Realtime channels (e.g. new ticket notes appear without a page refresh).

Cloudflare Worker (TanStack Start)

The Worker is the application's entire backend. Its entry point (`src/server.ts`) wraps TanStack Start's SSR handler. Business logic — tickets, workflow/approvals, predictions, compliance, analytics — is implemented as server functions the browser calls over RPC, with the user's Supabase access token attached as a Bearer header and validated before each handler runs.

Supabase

Provides Postgres, authentication, and realtime change feeds. The Worker accesses it through a separate service-role client that bypasses row-level security for backend reads/writes, while the browser accesses it through a public, RLS-scoped client for its own session and subscriptions. All application tables (tickets, notes, approvals, notifications, users) live in this one database.

Lovable AI Gateway

Receives ticket title/details from the Worker and returns a department and priority classification using Gemini 2.5 Flash; the same endpoint powers the chatbot. If the API key is missing or the call fails, the Worker falls back to a keyword-based heuristic classifier so ticket submission never hard-fails.

Resend

Sends all transactional email — account activation codes, password resets, and ticket notifications — triggered by the Worker immediately after the relevant database write.

pg_cron scheduler

A scheduled job inside Supabase's Postgres instance calls a public webhook route on the Worker once a minute. That route checks for admin messages a user haven't opened within a grace window and sends a follow-up email through Resend, so this path touches Supabase and Resend a second time, independent of any user action.

Communication Summary

Path	Mechanism	Purpose
Browser → Supabase	Direct client SDK call, RLS-scoped	Sign-in/session management; realtime subscriptions (e.g. live ticket notes)
Browser → Cloudflare Worker	TanStack Start server-function RPC, Bearer token attached	Ticket CRUD, workflow/approval actions, predictions, analytics, admin actions
Worker → Supabase	Service-role client, bypasses RLS	All backend reads/writes: tickets, notes, approvals, notifications, users
Worker → Lovable AI Gateway	HTTPS POST to ai.gateway.lovable.dev (Gemini 2.5 Flash)	Ticket classification (department/priority) and chatbot replies
Worker → Resend	HTTPS POST to api.resend.com	Activation codes, password resets, ticket notification emails
Supabase pg_cron → Worker	Scheduled HTTP call to a public webhook route, every minute	Flushes pending admin-message follow-up emails past their grace window

Technologies Used

Layer	Technology
Frontend	React 19, TanStack Router, Tailwind CSS 4, shadcn/ui, Radix UI components
Backend	TanStack Start server functions (RPC-style, co-located with routes)
Database / Auth	Supabase (PostgreSQL + Row Level Security, Supabase Auth)
AI Model / Gateway	Lovable AI Gateway — google/gemini-2.5-flash, with a keyword-based heuristic fallback
Email	Resend API (transactional email: OTP verification, ticket lifecycle notifications)
Charts / Analytics	Recharts (dashboard visualisations)
Reports	jsPDF and jsPDF-AutoTable (downloadable PDF report generation)
Forms & Validation	React Hook Form, Zod (schema validation on every server function input)
Hosting / Deployment	Cloudflare Workers/Pages (via Wrangler)
Development Platform	Lovable.ai / Bolt.new (AI-assisted development), Visual Studio Code, GitHub

Database Design

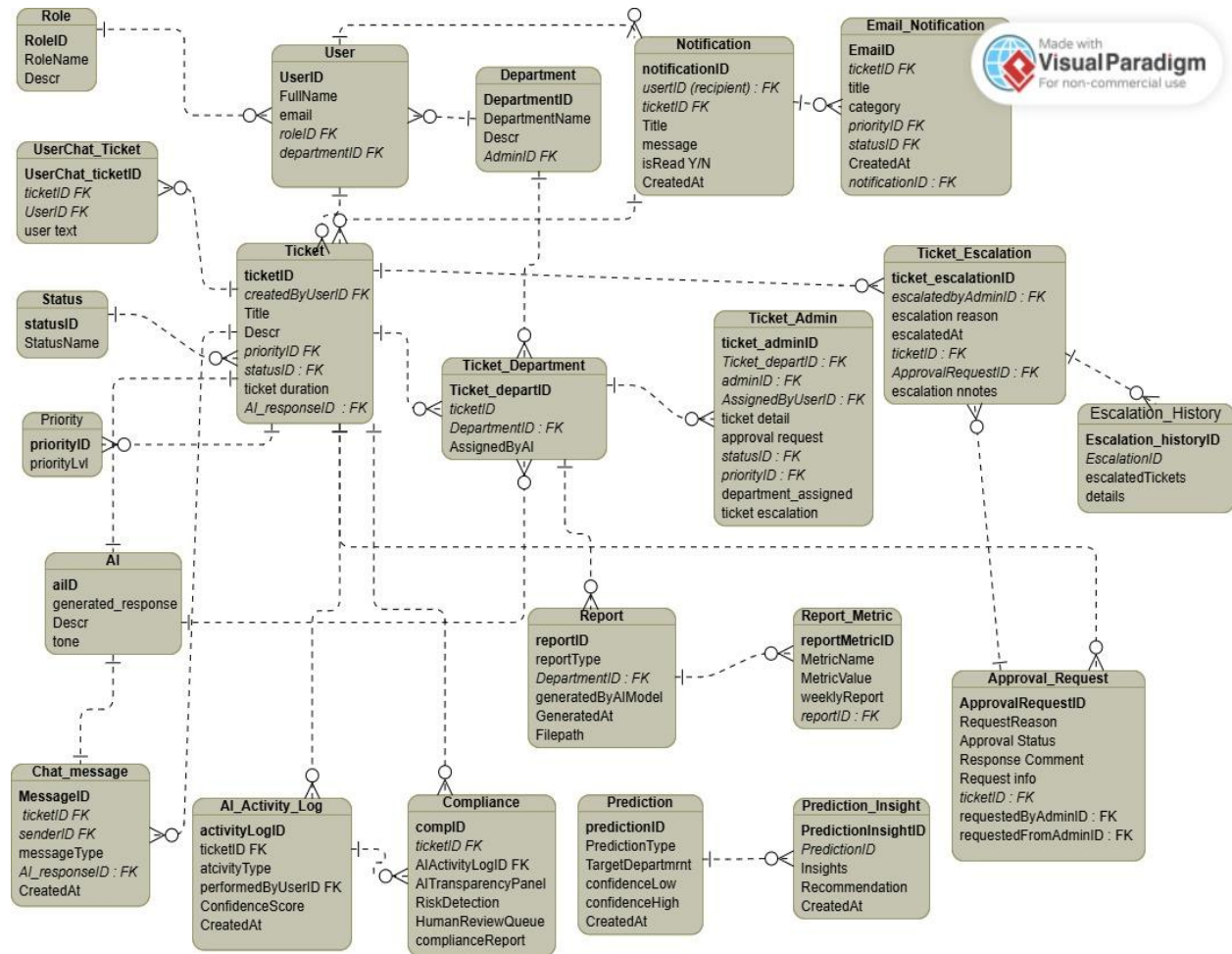


Figure 2: OpsAssist ERD

Entity-Relationship Diagram (ERD) Explanation

The Entity-Relationship Diagram (ERD) illustrates how data is organized and connected within the OpsAssist system. The **Ticket** entity is the central table because all support requests originate from it and connect to other system components.

- **Role** stores user roles (User, Department Admin, Super Admin), which determine system permissions.
- **User** stores user information, including their assigned role and department. Users can submit tickets, receive notifications, and communicate through the chat system.
- **Department** stores organizational departments such as IT, HR, Finance, and Operations. Each department manages multiple users and tickets.

- **Ticket** records support requests, including the title, description, priority, status, creator, AI response, and ticket duration.
- **Status** and **Priority** store predefined ticket statuses (Open, In Progress, Resolved) and priority levels (Low, Medium, High), reducing duplicate data.
- **Ticket_Department** links tickets to one or more departments, supporting many-to-many relationships and AI department assignment.
- **Ticket_Admin** records which administrator is responsible for managing a ticket.
- **UserChat_Ticket** and **Chat_Message** manage conversations between users, administrators, and the AI, providing a complete communication history.
- **AI**, **AI_Activity_Log**, and **Compliance** store AI-generated responses, activity logs, confidence scores, and compliance information to support transparency and monitoring.
- **Notification** and **Email_Notification** manage system alerts and email updates related to ticket activities.
- **Ticket_Escalation**, **Escalation_History**, and **Approval_Request** track ticket escalations, approval processes, and historical records.
- **Report** and **Report_Metric** store generated reports and performance statistics, while **Prediction** and **Prediction_Insight** store AI forecasts and recommendations to improve future support services.

Overall, the ERD separates data into related entities connected through primary and foreign keys. This structure reduces data redundancy, improves data integrity, and supports efficient ticket management, communication, reporting, AI automation, and decision-making within the OpsAssist system.

System Workflow

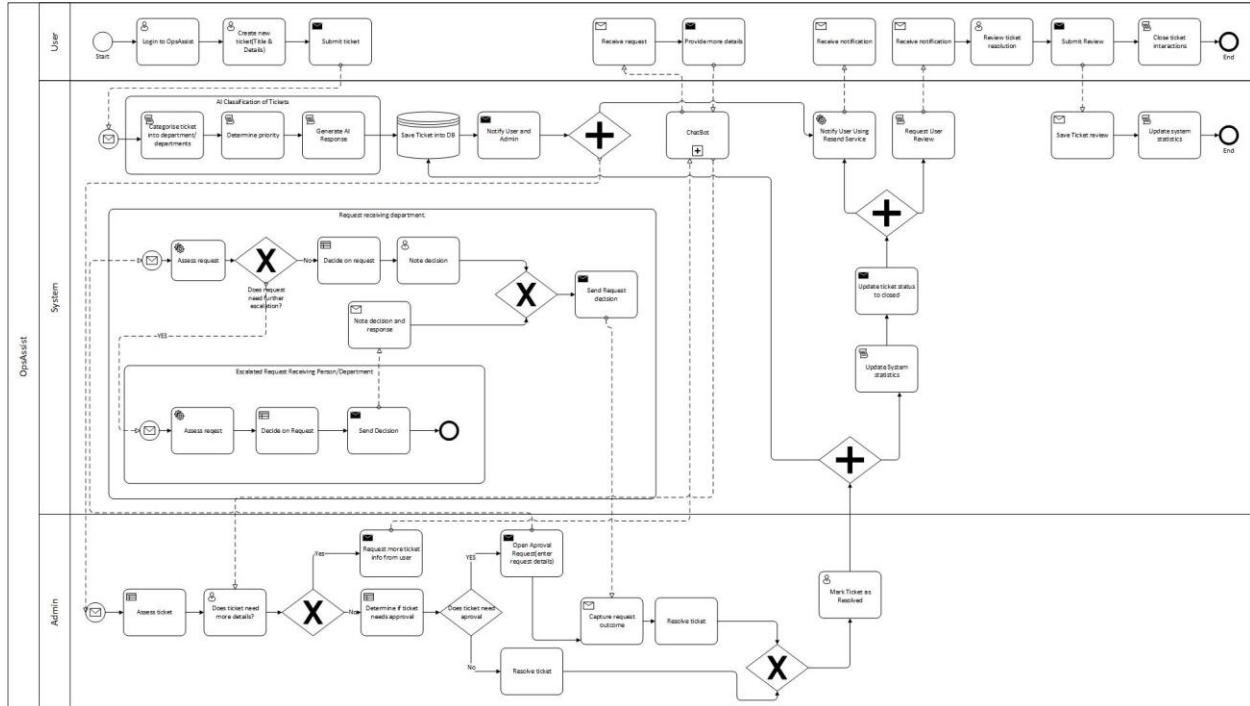


Figure 3: System Architecture Diagram

The following describes how a support ticket moves through OpsAssist from submission to resolution:

- 1. **Ticket Submission** — a user submits a ticket through the web form, providing a title and description.
- 2. **AI Classification** — a server function sends the ticket content to the AI model, which returns a department category and priority level; if the AI is unavailable, a keyword-based fallback performs the classification instead.
- 3. **Response Generation** — the AI response module drafts a professional first response based on the ticket's category and selected tone (Formal, Friendly, or Urgent).
- 4. **Storage & Logging** — the ticket, its classification, and the generated response are stored in the database, and the AI action is logged for transparency and audit purposes.
- 5. **Workflow Check** — the system checks whether the ticket type requires a multi-stage approval workflow (e.g. Finance or Manager sign-off). If required, approval stages are generated and the ticket pauses until each approver acts.
- 6. **Risk & Compliance Evaluation** — low-confidence classifications, conflicting predictions, or high-priority/low-confidence cases are flagged for human review with an assigned risk level.
- 7. **Analytics & Reporting** — once processed, the ticket contributes to the live analytics dashboard, weekly summaries, and forecasting calculations.
- 8. **Resolution & Closure** — once any required approvals are completed and the underlying request is actioned, the ticket status is updated to resolve/completed, with the full history retained in the workflow log.

AI Functionality

Artificial Intelligence is integrated into OpsAssist at several points in the ticket lifecycle, moving the platform from a purely reactive ticketing tool into a proactive, intelligent operations assistant.

AI Ticket Classification & Prioritization

On submission, ticket text is sent to an AI model (google/gemini-2.5-flash via the Lovable AI Gateway) with a structured prompt that returns a department category and a priority level, weighing urgency language in the request. This removes the need for staff to manually triage and route every incoming request.

AI Response Generation

Once classified, the system drafts a professional, category-appropriate response in a selectable tone. This gives support staff a strong first draft to review or send as-is, significantly reducing response time and improving consistency of communication.

Predictive Insights

A statistical forecasting engine analyses historical ticket data to estimate near-term ticket volume, detect trend direction, account for day-of-week seasonality, and flag likely workload surges — giving management the ability to plan staffing and resources proactively rather than reactively.

AI Governance & Risk Detection

Every AI-assisted action is logged with a confidence score and explanation, and the system automatically flags low-confidence or conflicting AI outputs for human review. This provides transparency into where the AI is being relied upon, and ensures a human remains in the loop for high-risk decisions.

Value to the IT Support Process

- Reduces manual triage time by automatically routing tickets to the correct department
- Speeds up first response through AI-drafted replies
- Improves operational visibility through AI-informed analytics and forecasting
- Builds trust and accountability through logged, explainable AI decisions and a defined human-review path

Challenges Encountered

Inefficient AI Prompting

Early in the project, prompts given to AI-assisted development tools (Bolt, New / Lovable AI) lacked sufficient detail, resulting in incomplete functionality, broken features, and inconsistent UI. This was resolved by adopting more structured, detailed prompting practices — clearly specifying functional requirements, expected formats, and technical constraints — which significantly improved the reliability of generated code.

Classification Accuracy & Keyword Overlap

Keyword-based classification initially misrouted some tickets where terms overlapped between departments (for example, "leave application" being misclassified as IT instead of HR). This was resolved by expanding and refining department-specific keyword datasets and testing against a wider range of example tickets.

Tone & Response Consistency

Some AI-generated responses did not consistently match their intended tone (e.g. friendly responses reading as too formal, urgent responses reading as aggressive). The team addressed this through iterative prompt refinement, more detailed response templates, and repeated testing across tone variations.

Dashboard Information Overload

As governance and compliance features were added, the dashboard became cluttered with the volume of information being displayed. This was resolved by reorganizing related information into dedicated, clearly labelled sections.

Team Coordination

With five team members contributing across frontend, backend, and AI integration simultaneously, coordinating decisions and avoiding conflicting changes became more difficult as the system grew in complexity. The team addressed this through more open communication, clearer discussion of trade-offs, and collaborative decision-making, prioritizing overall project progress over individual preferences.

Lessons Learned

Fanelesibonge Hlebani: This project strengthened my prompt engineering skills, as I learnt that clear, well-structured prompts produced far more reliable AI outputs than broad or vague ones. Working in a group can be tasking, but at the same time it's very important to work as a team towards achieving the same goal without putting each other down, putting our differences aside. Leading a team is no easy task but there's growth in that.

Mufhumudzi Moshapo: Building a shared, evolving system over eight sprints have taught me the importance of teamwork and consistent communication, since every new feature had to integrate cleanly with what the rest of the team had already built.

Lukholo Mabuto: I learnt that working through technical setbacks and disagreements as a team helps develop stronger problem-solving skills. I learned to break down complex issues, test assumptions, and find practical solutions for the group and together as a group.

Minenhle Sibiya: One of the key lessons I learned while working in a group is the importance of collaboration and open-mindedness. Even when I have my own suggestions or ideas, it is equally important to listen to and consider the perspectives of other team members. Being open to different viewpoints, rather than being stubborn, helps the team make better decisions and work more effectively together.

Asisipho Mbobo: I gained hands-on experience in AI integration, full-stack development, and designing intelligent system features while learning the importance of testing and continuous improvement. Beyond the technical aspects, I learned to work with individuals who had strong personalities, gradually becoming more confident in expressing my ideas, and I developed better presentation skills by communicating the project's objectives, features, and outcomes to others.

Future Improvements

- Replace the client-side admin password check with full authenticated, role-based admin access suitable for production use
- Add real-time/live dashboard updates instead of on-demand refresh
- Extend automated email notifications to cover more ticket lifecycle events (e.g. approval reminders, SLA breach warnings)
- Introduce more advanced machine learning models for classification in place of the current keyword fallback, to further improve accuracy on ambiguous tickets
- Add configurable SLA tracking and automatic escalation for overdue tickets
- Expand the compliance module with exportable audit reports for external review
- Support multi-language ticket submission and AI-generated responses
- Add a mobile-friendly / native app view for approvers to act on requests on the go

Installation & Running the Project

Suggested structure to complete:

- Prerequisites (Node.js/Bun version, accounts needed e.g. Supabase, Resend, Lovable AI)
- Cloning the repository on Visual Studio Code.
 - Press **Ctrl + Shift + P** to open the Command Palette.
 - Type **Git: Clone** and select **Git: Clone**.
 - Copy the repository URL from GitHub:
 - Open your GitHub repository.
 - Click the green **Code** button.
 - Copy the **HTTPS** or **SSH** URL.
 - Paste the URL into VS Code and press **Enter**.
 - Choose the folder where you want to save the project.
 - Wait for the repository to download.
 - When prompted, click **Open** to open the cloned project.

References

- Lovable.ai — <https://lovable.dev/> (AI-assisted development platform used to build and iterate the application)
- Lovable AI Gateway (google/gemini-2.5-flash) — used for AI ticket classification and response generation
- Bolt.new — <https://bolt.new/> (AI-assisted development tool used during early sprints)
- ChatGPT (OpenAI) — used for research assistance and debugging support.

Screenshots of the system

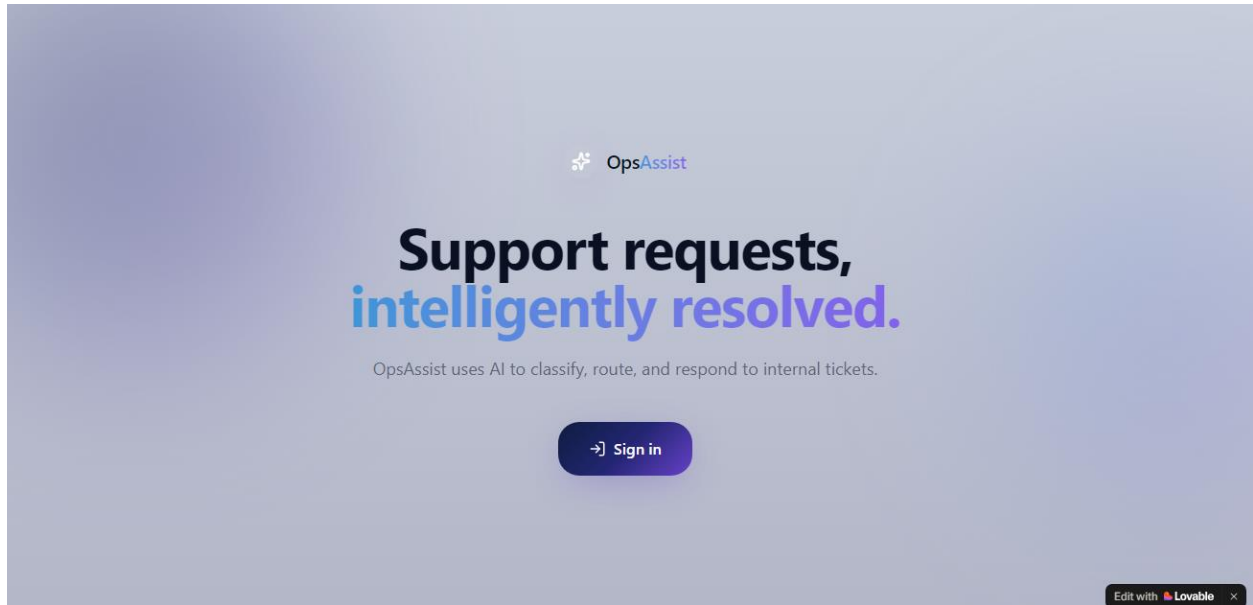


Figure 4: Landing Page

The landing page serves as the entry point to the OpsAssist ticket classification system.

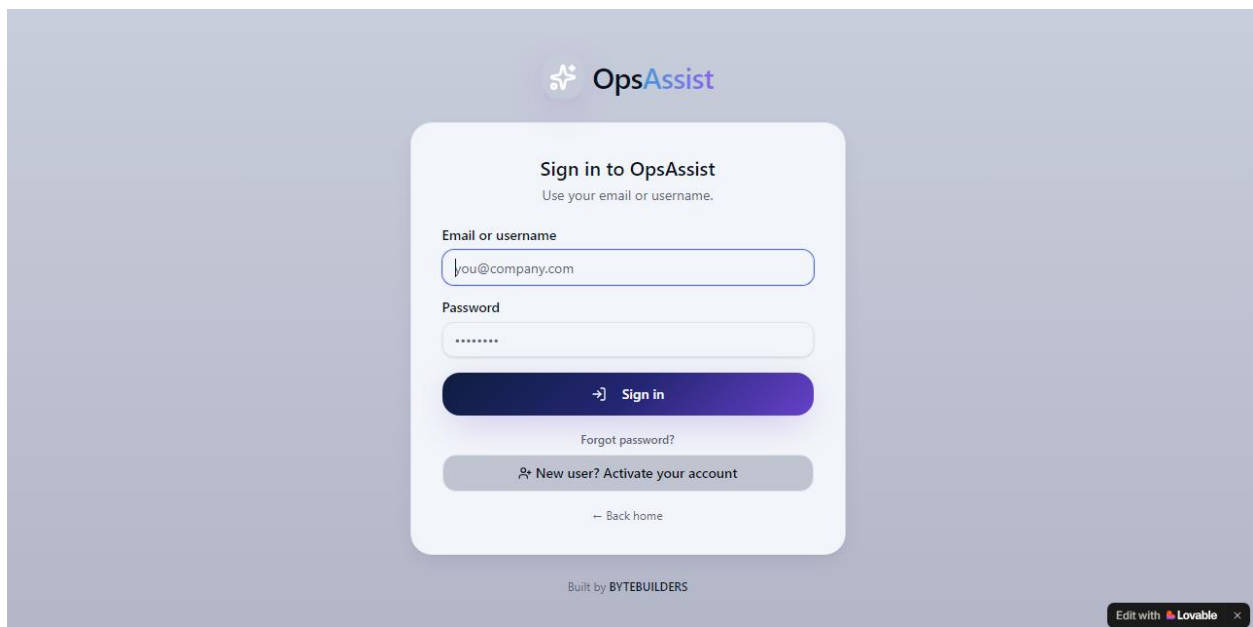


Figure 5: Sign In Page

The sign-in page allows registered users to securely access the system using their login credentials. Authentication ensures that only authorized users can access their respective dashboards.

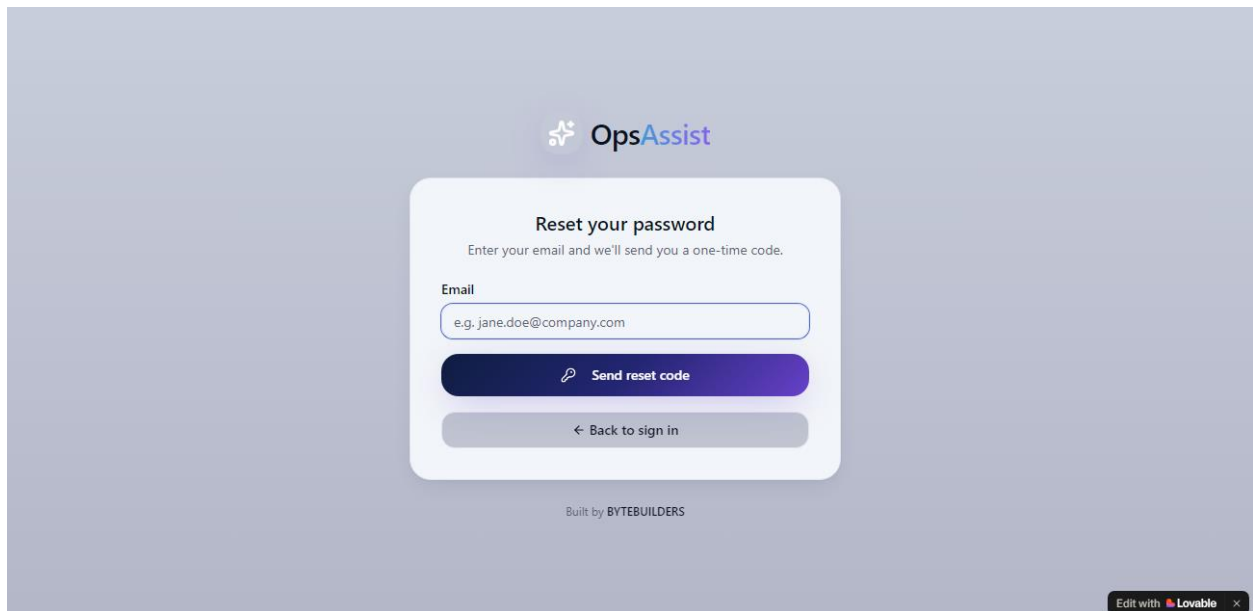


Figure 6: Password Recovery Page

This page enables users to request a password reset if they are unable to access their accounts.

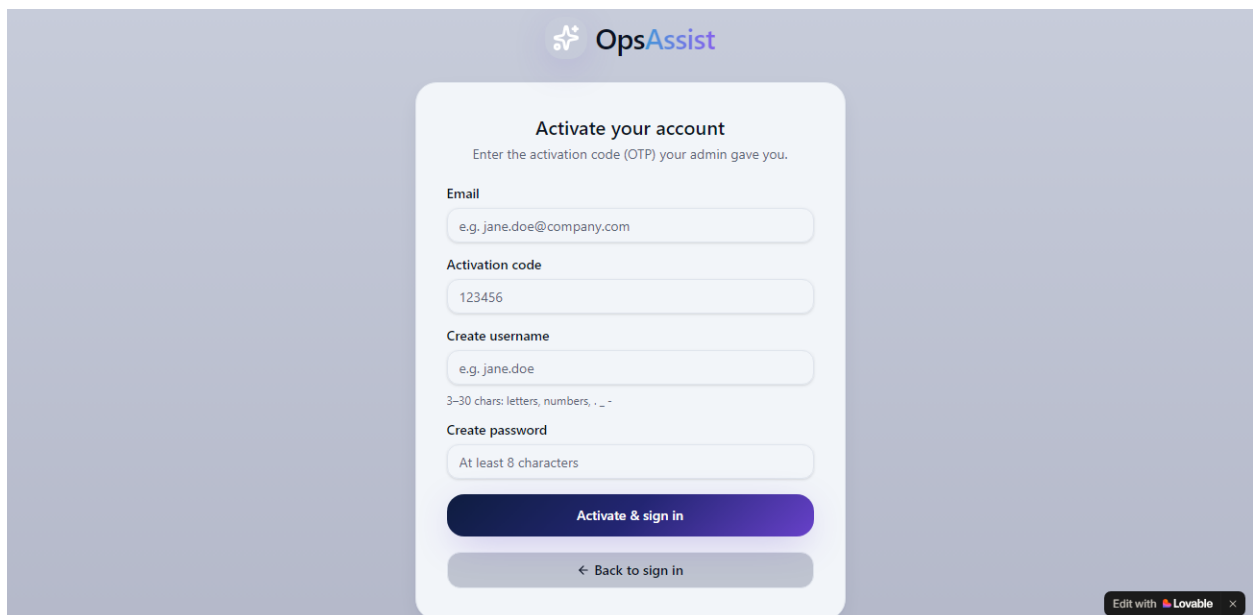


Figure 7: New User Account Activation Page

Newly created users activate their accounts by writing their emails, OTP/Activation code, creating their desired username and password.

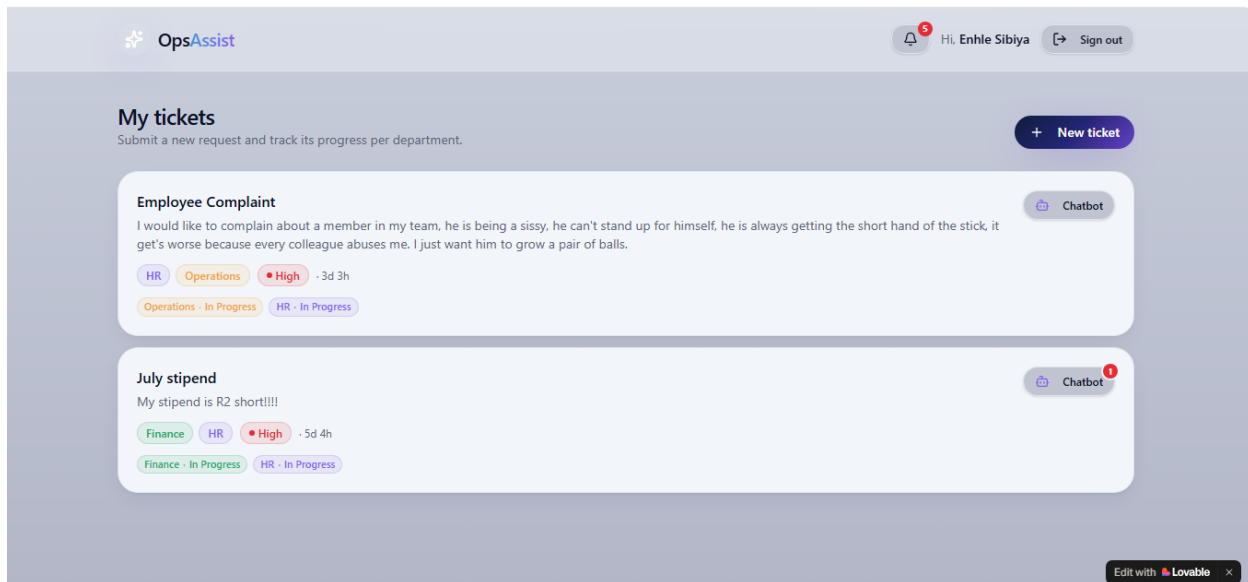


Figure 8: User/Employee Dashboard

The user dashboard provides employees with quick access to ticket submission, ticket tracking, and other system features. It serves as the primary workspace after logging in.

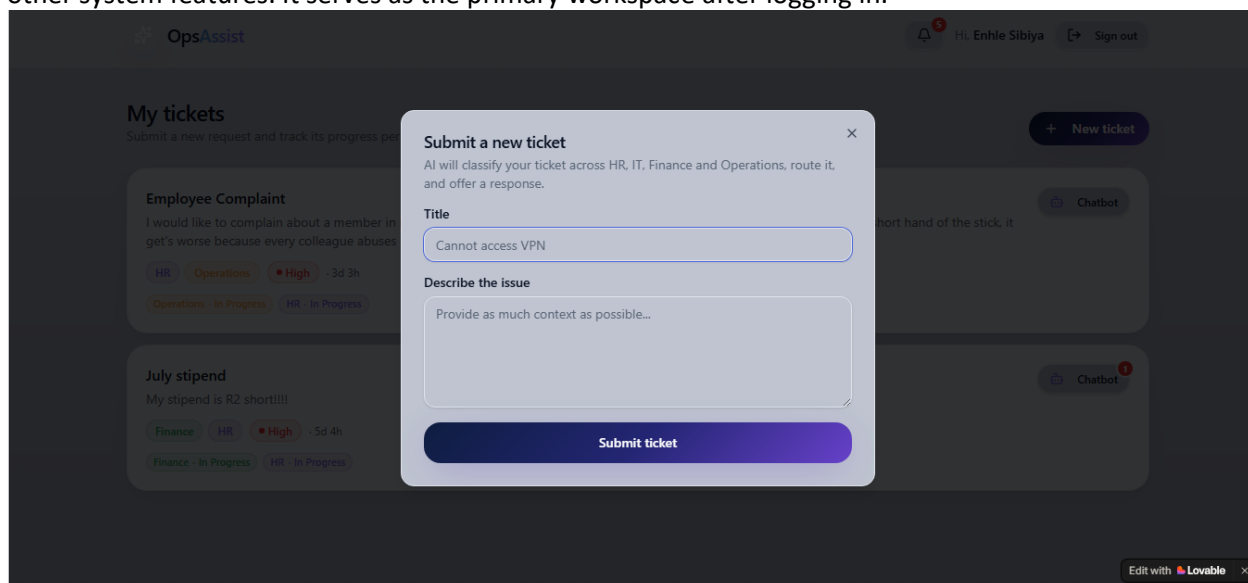


Figure 9: Ticket Submission Form (From the user dashboard)

Users submit support requests by providing a title and describing their issue. Once submitted, the ticket is forwarded to the AI classification component for processing.

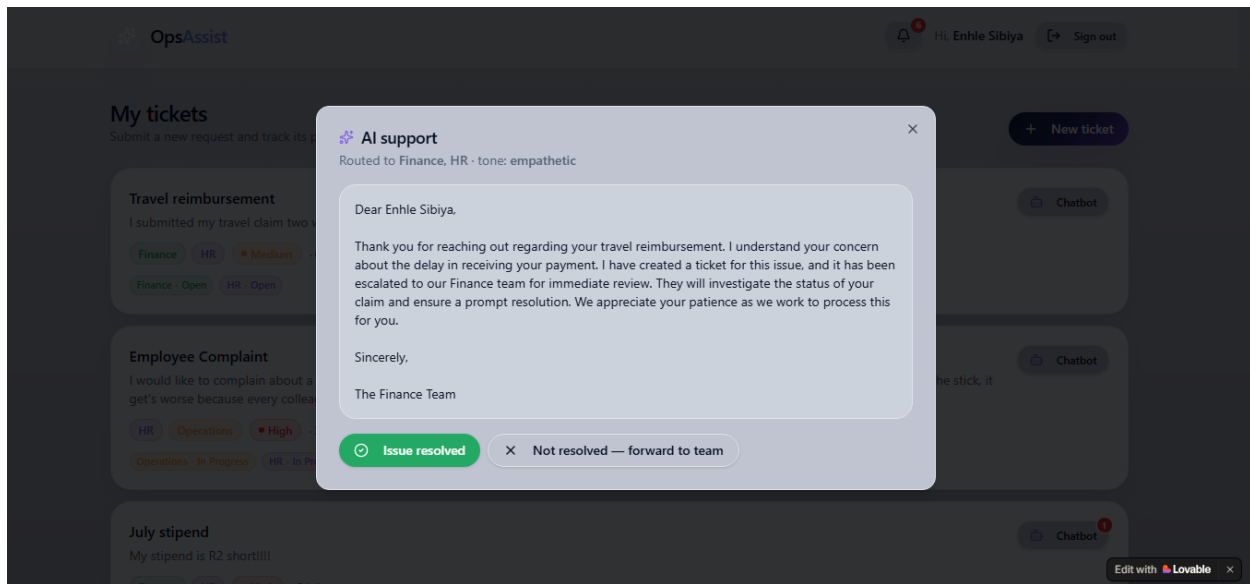


Figure 10: AI Classification Response

The AI analyses the submitted ticket and predicts the most appropriate department based on the ticket title and description. The AI also chooses the most appropriate tone to use as it responds. If the issue for the ticket was able to be solved by the AI response, the user clicks Issue solved then it is stored as history.

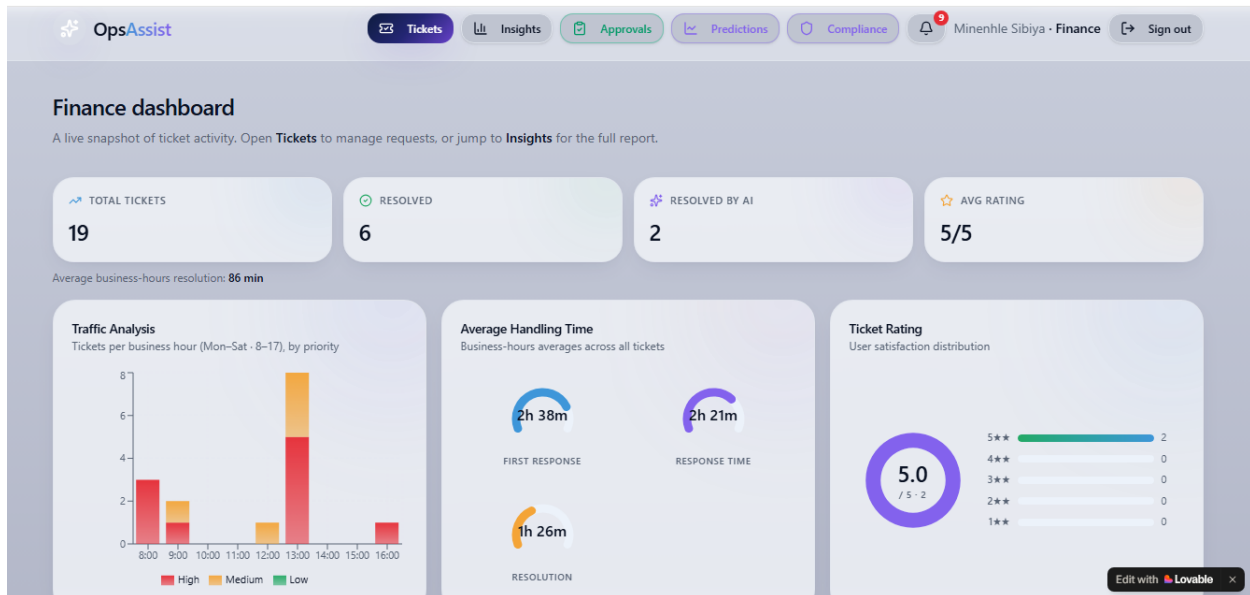


Figure 11: Admin Dashboard

The administrator dashboard serves as the central control panel for managing tickets, users, analytics, and governance features. It provides quick navigation to all administrative functions.

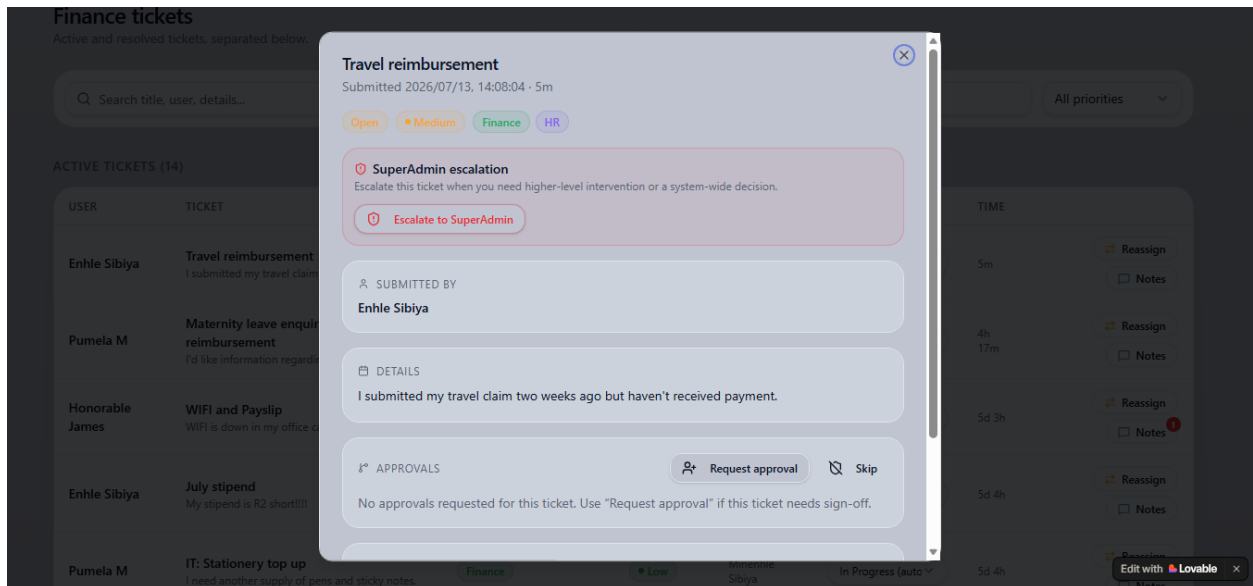


Figure 12: Assigned Ticket Details

Administrators can view tickets assigned to their department together with relevant information such as the ticket description, who it's from, and status of the ticket. This information assists administrators in resolving requests efficiently.

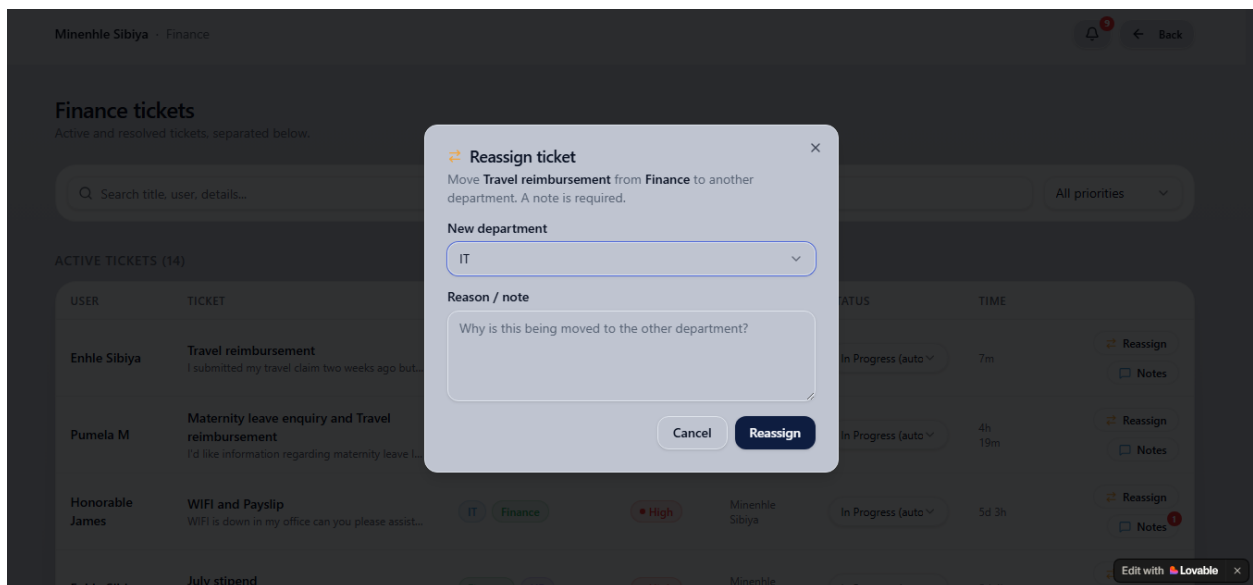


Figure 13: Ticket Reassignment Feature

If a ticket has been assigned to the incorrect department, administrators can reassign it to the appropriate department. This feature ensures that requests are handled by the most suitable support team.

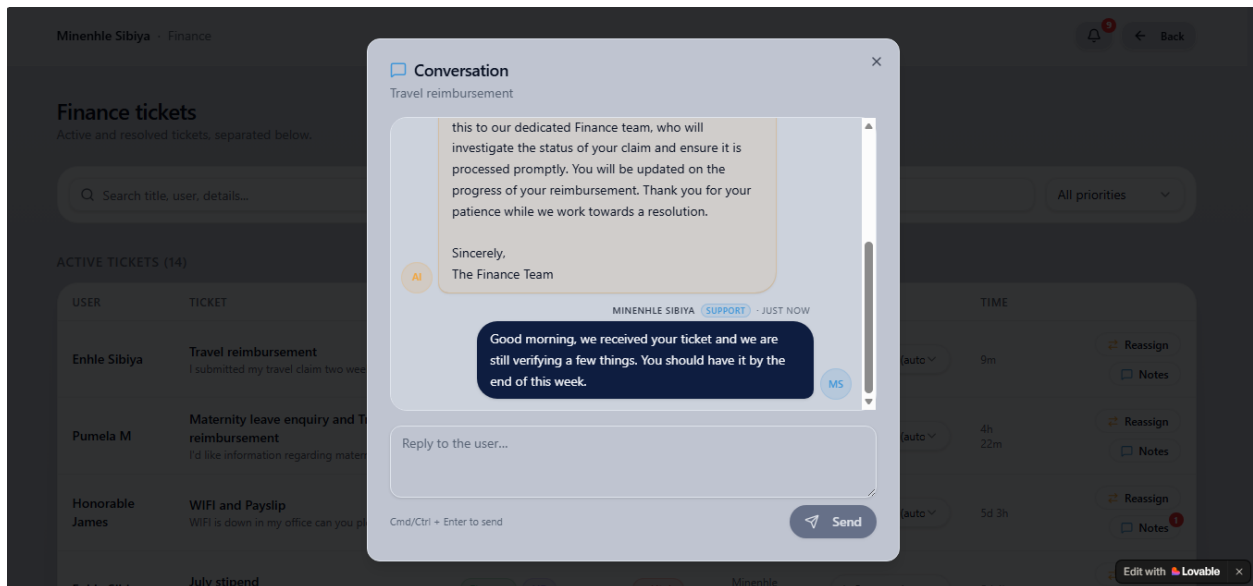


Figure 14: Communication Interface

The conversation page enables direct communication between the assigned administrator and the user who submitted the ticket.

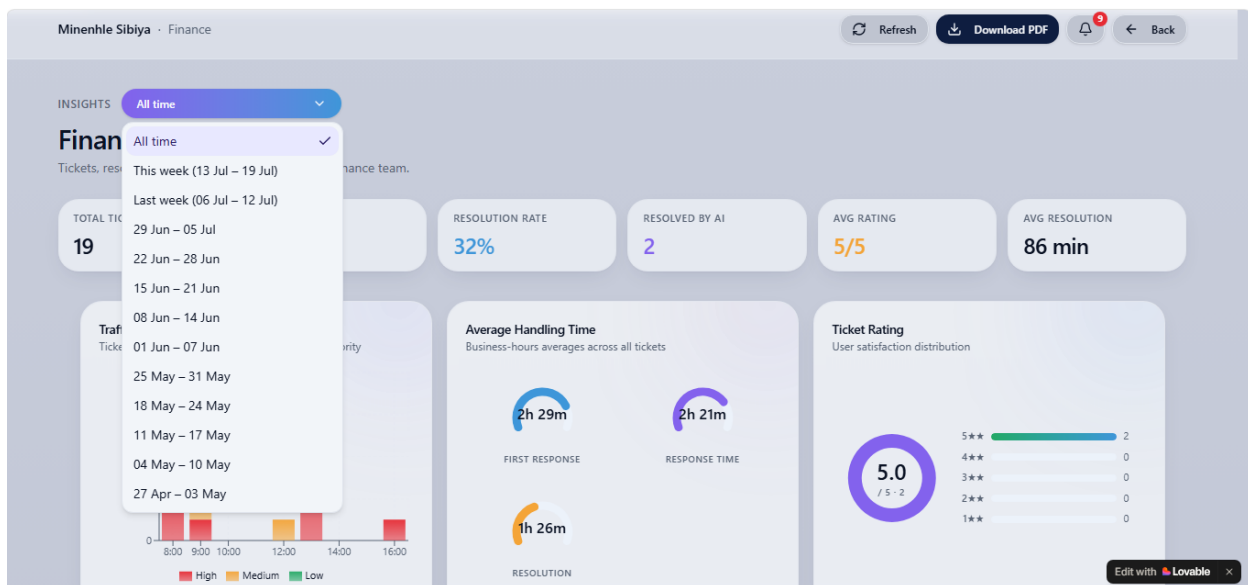


Figure 15: System Analytics Dashboard

The Insights dashboard presents visual summaries of ticket activity, including trends, AI narrative reports, and deep insights that include time analysis, common issues, suggested solutions to those issues, and recommendations. This could be tailored to a certain week or just general all-time insights. All this is downloadable and available in a pdf format.

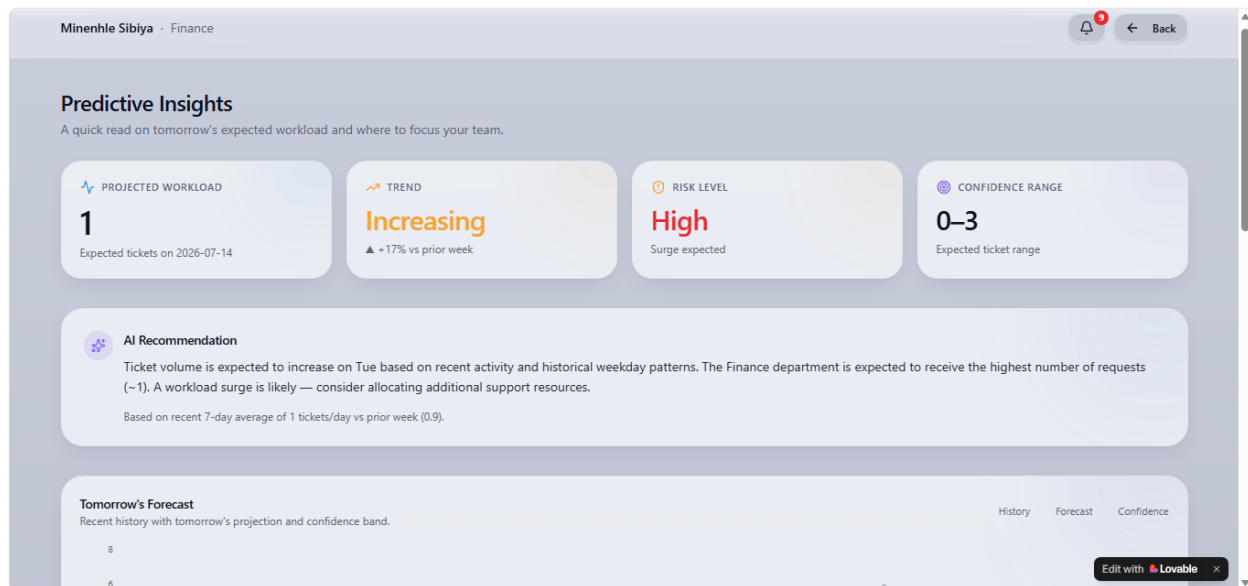


Figure 16: AI Predictions Dashboard

This dashboard uses AI to forecast the expected ticket volume for each department during the upcoming week. It also presents historical weekly trends and generates AI-driven recommendations to help administrators anticipate high workloads, allocate resources effectively, and implement proactive measures to reduce future ticket volumes.

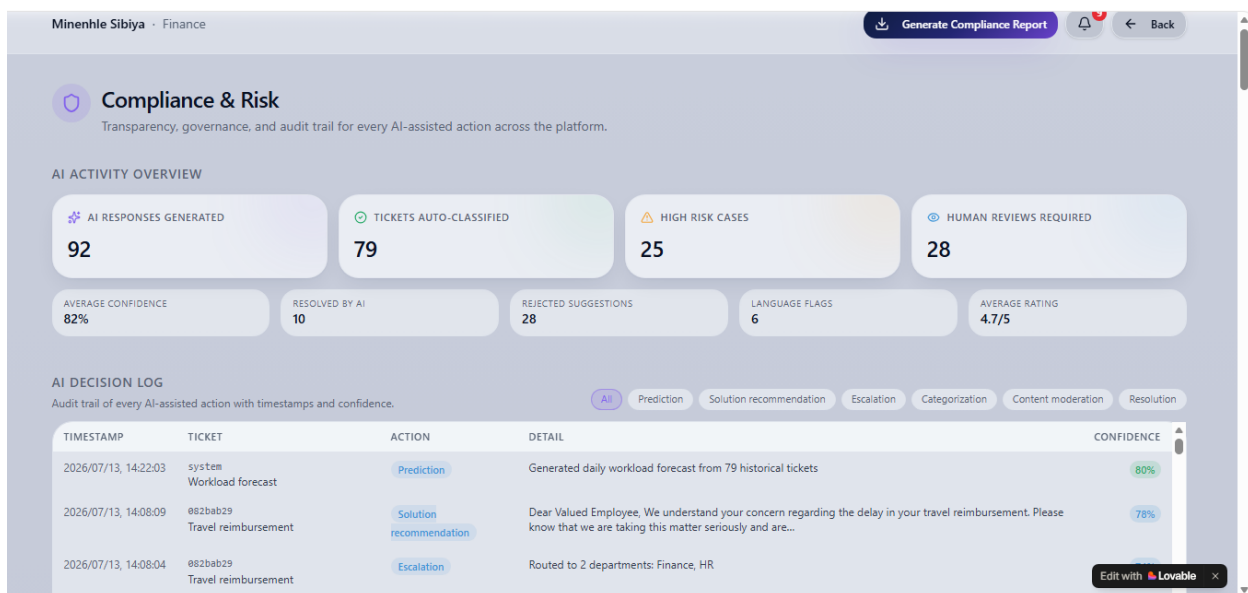


Figure 17: Compliance and Risk Dashboard

This dashboard provides governance features that support responsible AI usage within the system. It includes transparency information on automated decision-making processes and allows administrators to review classification outcomes, bias monitoring, and risk evaluation reports to promote accountability.

It's important to note that all the departments' admins have the same dashboards with information tailored to that specific department. The only notable difference is between the department admins and the Super administration who is able to add new users either as an employee or a new department admin. That specific page used to create/add new users is displayed at figure 18 below.

The screenshot displays the 'Users' management interface within the OpsAssist dashboard. The background shows the 'Admin dashboard' with a 'TOTAL TICKETS' count of 79 and a 'Traffic Analysis' bar chart. The 'Users' modal is open, featuring a form to create a new user and a table of existing users.

Users
Create Employees or Department Admins. OTPs are emailed automatically.

Form fields:

- Full name:
- Email:
- Account type:
- Buttons:

Table of existing users:

NAME	EMAIL	ROLE	DEPARTMENT
Enhle Sibiya	minenhlesthokomele@gmail.com	Employee	—
Fanele Hlebani	fanelehlebani@gmail.com	Employee	—
Lukholo Mabuto	lm.mabuto@gmail.com	Employee	—
Lukholo Mabuto	lukholo.m@icloud.com	Employee	—
Honorable Mabuto	mufhumudzi47@gmail.com	Employee	—

At the bottom right of the modal, there is a button labeled 'Edit with Lovable'.

Figure 18: Create New User Interface